

DeepEdge - Distributed Deep Learning Inference at Edge Devices

Final Year Project

Individual Report - 160477X

Table Of Contents

Table Of Contents	2
2 Introduction	4
2.1 Problem Statement	5
2.2 Proposed Solution	6
3 Literature Review	6
4 Research and Exploration	8
4.1 Research Overview	8
4.2 Research Challenges	9
4.2.1 Partitioning Inference Process at Convolution Sets	9
4.2.2 Communication and Task sharing	11
4.3 Model Selection	13
4.3.1 Open Neural Network Exchange	14
5 Methodology	14
5.1 Model Partitioning Schemes	15
5.1.2 Layer Wise Partitioning (LWP)	16
5.1.3 CNN Layers Partitioning Schemas	16
5.1.3.1 Input Wise Partitioning (IWP)	16
5.1.3.1.1 Architecture of the model	16
5.1.3.1.2 Input partitioning method	17
5.1.3.1.3 Overlapping Data Frame Calculation	18
5.1.3.1.4 Overlapping Pixel Calculation Algorithm	19
5.1.3.2 Kernel Wise Partitioning (KWP)	20
5.2 Implementation	22
5.2.1 High Level System Design	22
5.2.2 Model Extraction Unit (MEU)	23
5.2.3 Inference Engine (IE)	23
5.2.4 Communication demands	24
5.2.5 MEU Support for ONNX	25
5.2.5.1 Alexnet	25
5.2.5.2 Alexnet architecture	25
5.2.5.3 Steps for reading data inside model	26
5.2.6 Building a Raspberry Pi Cluster	26
5.3 Inference Task Distribution	27
5.3.1 Method 1	27
5.3.2 Method 2	28
5.3.3 Comparison between method 1 and method 2	29
6 Evaluation	29
6.1 Evaluation on a Custom Model	30
6.1.1 Model Architecture Introduction	30

6.1.2 Input size vs Inference time (Three Devices)	31
6.1.3 Input size vs Inference time (Four Devices)	33
6.1.4 Device count vs Inference Time (Ethernet)	34
6.1.5 Device count vs Inference Time (WIFI)	35
6.2 Evaluation on VGG 16 Model	36
6.2.1 Model Introduction	36
7 Conclusion/Summary	44
References	46

1 Abstract

Deep Neural Networks (DNNs) and Convolutional Neural Networks (CNNs) are widely used in IoT related applications. However, inferencing pre-trained large DNNs and CNNs consumes a significant amount of time, memory and computational resources. This makes it infeasible to use such DNNs/CNNs on resource constrained edge devices. In this research we introduce an approach to solve this issue by processing the inference tasks using two or more edge devices. A novel approach of distributing the inference process among multiple edge devices while minimising the network overheads due to communication among devices, a task sharing mechanism among working devices and idle devices in the network and a way to convert pre-trained models to a separately executable model format is introduced through this research. Evaluation of the proposed solution showed that it can obtain upto 60% inference speedup with ethernet communication medium and upto 40% inference speedup with WiFi communication medium while preserving the accuracy level at 100%. We conclude that distributed deep learning inference at resource constrained edge devices is successful with a huge speedup gain compared to doing the whole inference process at one edge device.

2 Introduction

Deep neural networks are becoming an important tool in modern computing applications. Deep Neural Networks (DNNs) and Convolutional Neural Networks (CNNs) are heavily researched domains and are widely used in large scale data processing applications due to their higher classification accuracy compared to other methods[13]. Lot of data and computation power is required to train/use these models. Edge computing and IoT applications have emerged with the requirement of collecting and processing large sets of data locally within devices, thus, ensuring privacy and low latency. However, these edge devices are resource-constrained and have less computational power, memory, and energy compared to modern computers and servers. DNN/CNN has two phases which are called training and inference. Training a neural network requires a large set of data and huge computational effort. Once the model is trained and ready it can be fed previously untrained input data and will return results with identified or classified patterns in the input data. This phase is called inference. Even Though it does not require huge computational power compared to training, executing inference tasks in resource-constrained edge devices is still a challenging task [1].

Performing inference tasks of deep learning applications on edge devices ensure the privacy of the input data and prevent latency and availability issues compared to a cloud solution. As most edge devices are resource-constrained (eg: memory, computational) they cannot store and execute an entire Deep Neural Network(DNN). According to paper [12], One possible solution is to distribute the DNN across multiple edge devices. In this research, we are focusing on the execution of the DNN inference part in resource-constrained edge devices like Raspberry Pi boards. We intend to research and propose a novel approach to partition and distributively execute inference calculation on multiple edge devices that are connected in a local network. Furthermore, we plan to investigate how to efficiently reduce the communication demands among the devices.

2.1 Problem Statement

IoT devices have been facing the issue of low resources (e.g. computational capacity, memory, energy) since the beginning of the IoT era [10]. Thus, IoT applications that need to use computationally intensive technologies like augmented reality, video processing, cognitive assistance, and machine vision are unable to get the best performance and real-time experience with these constraints. Performing inference tasks of deep neural networks on IoT edge devices requires considerable computational power and memory space which are usually limited on IoT devices. Relatively high-end IoT boards such as Raspberry Pis also have limited resources eg: 1GB - 2GB RAM, 1GHz - 1.5GHz processor, etc. (Depends on the versions.) Therefore, storing and executing a large complex Deep Neural Network like Convolutional Neural Networks in a single device is an infeasible task. Even a medium scale CNN may present intolerable delays in executing inference calculations in these devices. The most common solution to this problem is cloud-based computation which introduces availability, privacy, and latency issues. Local computing ensures the privacy of sensitive data and results in low latency. For fully connected neural networks with a large number of layers and neurons, computationally intensive convolutional layers, the resources of a Raspberry Pi device isn't enough and increases latency.

Deep Learning Software Framework	MobileNet v1 (ms) Benchmark value	MobileNet V2 (ms) Benchmark value	Inception model that we tested (s)
TensorFlow Lite	271.5	379.6	7
TensorFlow	480.3	654.0	8

Table 1: A comparison of image classification time taken on a Raspberry Pi device

As shown in Table 1 above, Tensorflow optimized models like MobileNet v1 and v2 have acceptable low time consumption to complete execution, but we have tested a custom model (Inception model) an object detection library on our Raspberry Pi 3 B V1 setup, and it took an unacceptable amount of time compared to the Tensorflow given benchmark values [11]. We identified a lack of conducted research in this area to provide model partitioning and distributed execution in resource-constrained IoT devices (etc. raspberry pi, mobile devices..) considering computational, memory, and communication demands. So our problem is to find a more efficient way to run large Deep Neural Networks and improve the overall performance of distributed Deep Learning Inference on low resource IoT edge devices.

2.2 Proposed Solution

The final outcome of this research was to propose a novel approach to build communication and memory aware partitioning and distribution schemes for convolutional neural networks among resource-constrained edge devices. Furthermore, a proof of concept prototype for distributed DNN/CNN execution is also planned to be implemented.

- a. Two main partitioning methods that can be applied to partition convolutional layers.
- b. Model parallelism based partitioning scheme for fully connected neural networks.
- c. Task distribution and task sharing mechanism among suitable resource-constrained edge devices. (Eg: Raspberry Pi, mobile devices)
- d. Proof of concept DNN/CNN partitioning and distributed processing application using python.
- e. Evaluate the performance of the developed system.

3 Literature Review

Improving Deep Learning inference performance to overcome challenges it has faced is an intensively researched area [1]. Inference is computationally costly when it comes to high dimensional inputs. Therefore, a lot of calculations have to be done on that input data. One common solution was to use cloud computing as a solution, as they can use cloud resources for computation. But moving the data from the source to cloud introduced some challenges such as latency, scalability and privacy [1]. When it comes to real time applications, latency incurred by

additional queuing and propagation delays of the network is not acceptable. Unpredictability of the remote cloud server status is also a problem. Uploading all the data from the source to the cloud is inefficient too. Sometimes there may be privacy issues when uploading data. Therefore, to avoid those challenges edge computing came into the play. Edge computing reduced the latency by doing computations in close proximity to the end devices. A hierarchical structure and a local edge server can be used to avoid privacy issues. But edge computing also introduced new challenges. One of the main challenges is using the less powerful edge computing devices for the high resource requirements in deep learning applications. An edge device may be an edge server with a GPU or a mobile device or it may be even a Raspberry Pi device. Another challenge was to coordinate with edge devices to produce an output.

As mentioned in [1], there are three approaches for Deep learning inference in edge computing scenarios.

1. All computations done on the end device.
2. Edge server computation.
3. Computation across edge devices.

In the first method, the model is located at the edge device. As the edge devices are not capable of doing heavy computation and memory intensive tasks, those models need to be designed with less parameters. Otherwise, later the parameters should be compressed or unnecessary parameters should be removed. In DeepIoT [2], the authors present a pruning method for DNN models used in IoT devices. But this may have an impact on the results by reducing the model accuracy. Another sub method of this is where application-specific hardware is used for solutions in order to speed up the process and save energy [1].

Though all those methods are considered, the first method is still challenging because of the resource limitations such as power, computation and memory [1]. Because of that some researchers tend to use the second method which involves an edge server. As done in the deep learning framework presented by the authors of paper [3], where they offload the whole DNN to the edge server for computations. But sometimes it is helpful to do data preprocessing to reduce the communication time with the edge server. In [4], they use removing blurry images, and cropping to take only the required parts of images as preprocessing techniques. A main problem faced here is resource management in the edge server as multiple end devices may be using the same shared edge

server. Because of that tradeoff between latency, accuracy and other performance metrics need to be considered. Some solutions such as Mainstream [5] use transfer learning, where common lower layers of the DNN model are used for multiple applications to achieve the above mentioned tradeoff.

Although the second method increases the speed of inference, there is a third method more efficient than the above method, where intelligent offloading is considered. Solutions done in this method face challenges such as taking a decision whether to offload the entire DNN or not (depends on size of the data, hardware abilities, DNN model, network quality), and which fractions of a DNN computations should be offloaded [1]. When using DNN partitioning, some solutions have used layer-wise partitioning as in Neurosurgeon [6] and some have used input-wise partitioning as in MoDNN [7] and DeepThings [8]. Input-wise partitioning also has a problem of increased data dependency. Some solutions such as DDNN [9] are using a hierarchy of cloud, edge servers and end devices to model partitioning. Cloud is used only if total processing time is reduced, because of the available powerful resources and increased privacy risk.

Based on our literature survey, we saw a lack of research done in the area of distributed deep learning inference across multiple helper edge devices without using a remote cloud. Latency and task management are the main issues which are not much addressed in the existing solutions. Latency is caused by less efficient load balancing heuristics and dynamic task sharing systems which we will be improving in our solution. Using fine-grained partitioning of the DNN executions across edge devices, considering their resource availability and network condition seems like a promising method for our solution according to the research.

4 Research and Exploration

This section will provide an overview of the research exploration done from the start of the project to the final results. And also this chapter will include the challenges faced and how we overcame them.

4.1 Research Overview

We started with initial literature review and started gathering knowledge about the stated problem and the domain of the convolutional neural network and resource constrained devices like raspberry

pi. There we found various researches done to improve the deep learning inference performance at edge devices. Based on the literature review, we identified that offloading deep learning inference computation across edge devices as a suitable approach. For offloading part of computation to other devices, the neural network needs to be partitioned in a way that it can be processed separately on multiple devices. Therefore our first objective was to propose a suitable partitioning schema for neural networks.

Our initial target was to distribute Fully Connected Neural Networks (FCNNs) among multiple devices. However, later we decided to research the possibility of distributing Convolutional Neural Networks (CNNs) as CNNs tend to require more powerful devices to run the inference task. These CNNs typically take longer inference time compared to FCNNs. Our research exploration was separated into two phases whereas in the first phase we wanted to propose suitable distribution schemas for FCNNs and CNNs. And in the second phase we intended to implement a proof of concept distribution application and evaluate the performance improvement of the inference execution task.

4.2 Research Challenges

4.2.1 Partitioning Inference Process at Convolution Sets

Convolution sets are the middle layers in CNNs. These layers include so many calculations which increases the latency of the process. We experimented model parallelism based partitioning methodology for partitioning the process at Convolution Layers in resource constrained edge devices. This partitioning should have the ability to distribute the workload among a varying number of worker devices depending on their availability. There will be two configuration options for the distribution method.

1. When the number of worker devices in the network is fixed.
2. When the number of worker devices in the network is dynamic.

Therefore the partitioning schema should be able to partition the inference process depending on the availability of worker nodes.

In our research we used images as the input data to CNN models. These images have height and width dimensions and also they can have some number of channels. If the image is a grayscale image or a single color image, then the number of channels will be only 1 but if it's a color image then there will be 3 channels whereas Red channel, Green channel and Blue channel. These images are seen by computers as matrices of numbers representing some color density value on each pixel.

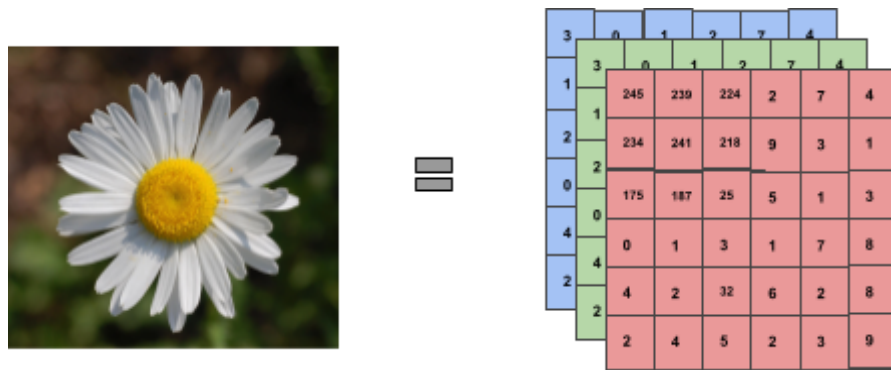


Figure 1: Input Data Representation

The input data can be considered as a mathematical representation of the density of each pixel value on each channel. This mathematical representation can be used to apply other mathematical operations. Therefore this input image is considered as a 3 dimensional matrix. These matrices can be represented as Arrays in Arrays or in Arrays or as multidimensional numpy arrays in python. This multidimensional array of data can be stored as tensors in various neural network frameworks like Tensorflow and PyTorch. These tensors are used to apply operations like convolution, pooling and flattening etc.

As the initial phase we tried to identify what are the possible partitioning methods that can be applied to partition convolutional neural networks layers. Typical convolutional neural network architecture can be introduced as shown in Figure 2.

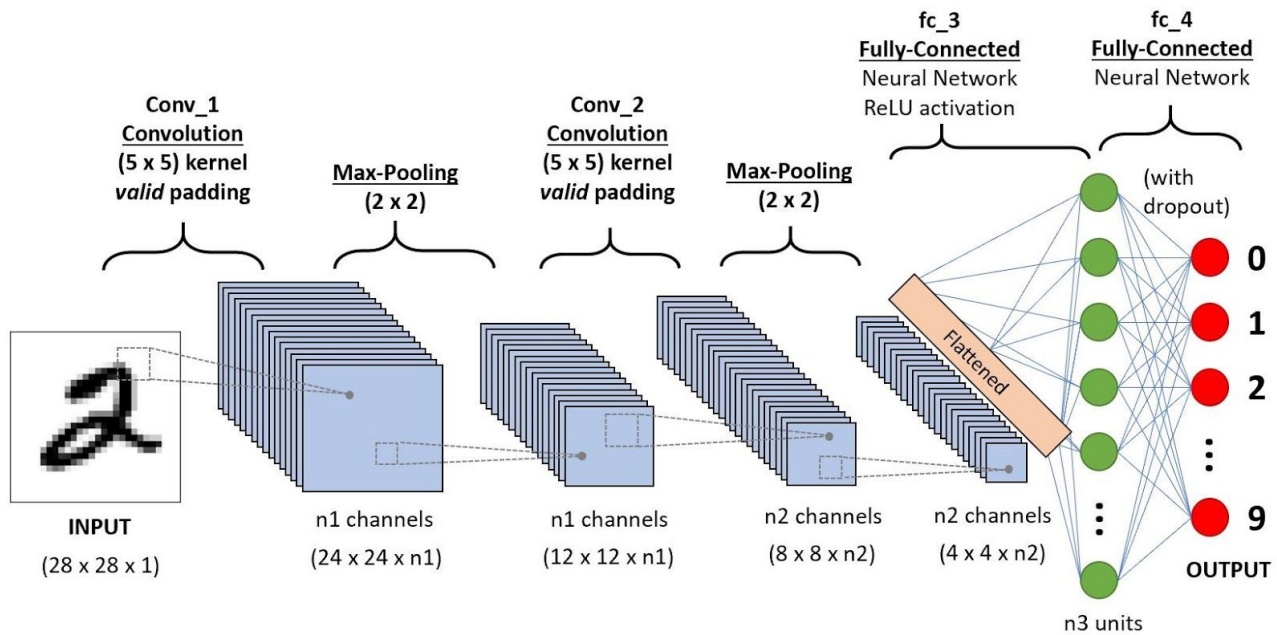


Figure 2: Convolutional Neural Network Architecture

As shown in the above figure there are multiple layers of convolutional layers with multi channel input of varying dimensions and multi channel outputs with varying dimensions. There are a fixed number of kernels that are used for the convolutional operation on each convolutional layer. Also it's known that the initial part of a CNN is mostly dominated by the input data. But the later parts of a CNN are mostly dominated by kernels. We had to consider each of these aspects when partitioning the inference process between devices.

4.2.2 Communication and Task sharing

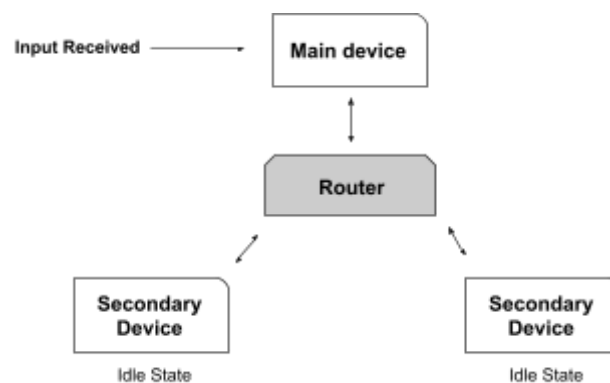


Figure 3: Raspberry Pi Communication Cluster

As shown in the above figure (Figure 3) the main device will receive input data to the model. In this case we assume that there are two other devices on the same network which are in idle state or doing less computationally expensive tasks. These secondary devices also can be changed to a main device dynamically when it receives an input. All three devices have pre-trained model and the partitioning and sharing services running in the background.

The main device which captured the input data will run the partitioning program to partition the neural network and input data to separately executable tasks and it will add them to a task queue. Then the main device will start the inference execution by taking one task by one from the queue. Meanwhile, other idle devices on the same network will ping the main device that they are available to participate in the task. The main device will then start sending a set of tasks from the queue to secondary devices. When the secondary devices have finished the calculation they will update the main device with results and request more tasks from the queue until it is empty or the execution is interrupted due to its own requirements of executing its own process.

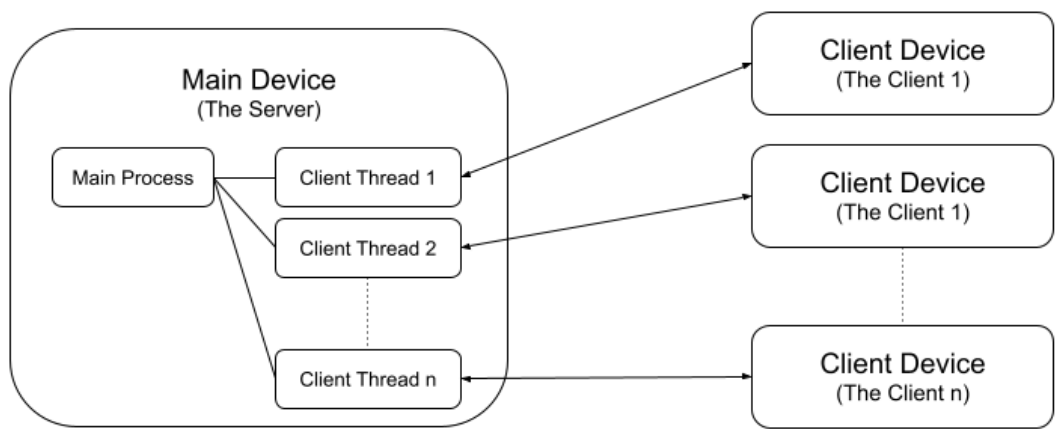


Figure 4: Threaded Communication Handler

Whenever there is a task to be done on a client device. Communication handler will take the task from the task queue and send it to the client device with the required data and the part identifier to be processed. And the client device connection will be removed from the AVAILABLE_DEVICES queue. Once a client receives a task it will start processing it immediately and stop sending !PING requests any server. After completing the process on the client device it will send the result with the part identifier.

4.3 Model Selection

We searched for computer vision models which are widely used by developers and researchers. We came up with multiple model collections aka modelzoos available on the internet related to computer vision domain.

There we came with few kinds of deep learning frameworks which are commonly used to build and run these models. For example,

1. Tensorflow (<https://www.tensorflow.org/>)
2. TensorFlow Lite (<https://www.tensorflow.org/lite>)
3. Caffe (<https://caffe.berkeleyvision.org/>)
4. Pytorch (<https://pytorch.org/>)
5. MXnet (<https://mxnet.apache.org/>)
6. Keras (<https://keras.io/>)
7. Darknet (<https://pjreddie.com/darknet/>)
8. ONNX (<https://onnx.ai/>)

Among these frameworks Tensorflow , Keras and Pythorch are most widely used.

Each of these frameworks provide many computer vision related models. And those models can be trained and used for the inference using the inference engines of those frameworks.

Most of these model developers/original researchers provide pre trained models which users can directly use in the inference process without doing their own training. These trained models have two separate parts which can reside in a single file or which can be separate two files

1. Model architecture
2. Weight file

Model architecture file contains the details like what are the layers in convolutional neural networks, what are the dimensions of filters , how the layers are ordered, architecture of the fully connected layers, etc.

Weight file contains the weight values of each filter and neurons in the network.

4.3.1 Open Neural Network Exchange

While searching for a method to support as many models, we found Open Neural Network Exchange aka ONNX. It is an open standard for machine learning interoperability. ONNX can be used to convert pre-trained models into onnx format. Therefore, if we can support ONNX on our framework we can use it as a gateway to use many other pre-trained models. Finally we came up with the idea of developing our own model converter tool which can read and convert external models into our own format which is fully optimized for our distributed inference engine.

5 Methodology

This research project was primarily targeted on achieving two main goals as described below.

First Goal

In the first phase of this research our main objective was to find a novel approach to build communication and memory aware partitioning schemes for convolutional neural networks. In this phase we tried to find different ways that we can partition a convolutional neural network into separately executable smaller computation tasks. When partitioning the neural network we need to be aware about the memory and communication requirements that will arise when we apply this partitioning schema to distributedly execute a convolutional neural network among multiple edge devices.

There we need to be able to minimize transfer size and to minimize the number of times that these data transferring happens among devices. Thus we can decrease the amount of network calls and the size of the transfers will be reduced resulting in lesser time wastage on communication.

Second Goal

The second phase of the research is to build proof of concept applications using proposed partition schemas and evaluate the performance improvement of inference execution in resource constrained edge devices. This evaluation step helps us to select the best option among proposed partitioning schemas in the first phase.

Initially we decided to use low level language like C to implement the system hoping it can provide higher performance. However considering the highly optimized library support like numpy library, we may not be able to obtain such performance in C language as we have to implement functionalities that are provided by these optimized libraries. Therefore we decided to use python for the implementation of the proposed proof of concept system.

5.1 Model Partitioning Schemes

Convolutional operation is the main and most costly operation in a convolutional neural network. This convolutional operation is a mathematical calculation done on matrices containing some data about the input data or model related variables like weights and biases.

Convolution on two matrices can be explained as follows.

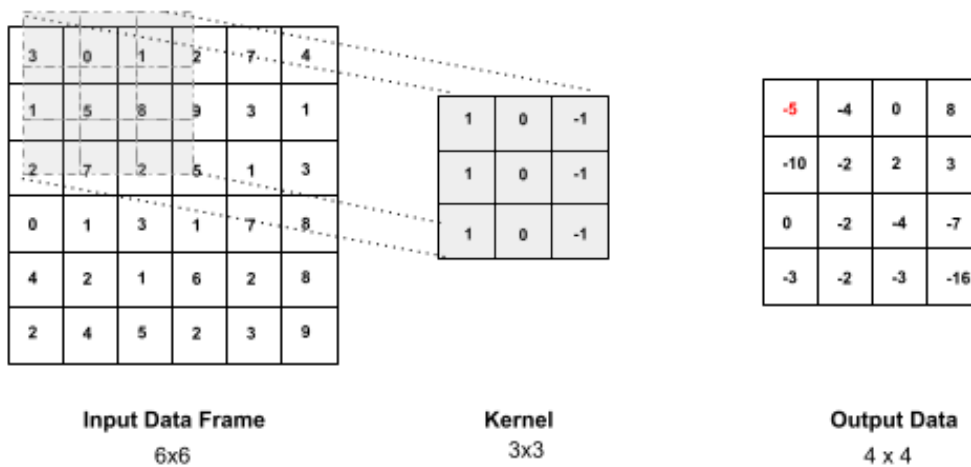


Figure 5: Convolutional Operation

In the above example Input data frame dimension is 6 x 6 and kernel size is 3. The stride is one. Convoluting the input data frame with the given kernel produces the shown output data. This calculation is done using matrix operations on computer processors. Based on the nature of convolutional neural networks we came up with two major partitioning schemas names as Input Wise Partitioning (IWP) and Kernel Wise Partitioning (KWP). I will introduce the methodology for IWP, which I mainly worked on.

5.1.2 Layer Wise Partitioning (LWP)

Layer Wise Partitioning is a widely used partitioning technique in many deep learning applications. But LWP will only be effective if there are a large number of worker nodes and if it processes data as a batch. Most available inference engines use this method as it is simple and flexible regardless of the model architecture. Also this method is effectively applicable in GPU or multi core processing.

Device 1 will work on the 1st layer and pass results to the second device to work on the second layer. Device 1 will take the next input and start working on the 1st layer while Device 2 will complete the first task and pass to the 3rd device. This process will continue until each node is working on a specific layer of different data.

But we decided not to use this method as it does not suit our requirement.

5.1.3 CNN Layers Partitioning Schemas

Our goal is to partition CNN layers into two or more independently executable partitions. As described earlier the two partitioning schemas that we have proposed

1. Input Wise Partitioning (IWP) (I mainly worked with this schema)
2. Kernel Wise Partitioning (KWP)

5.1.3.1 Input Wise Partitioning (IWP)

When dividing the input data for the inference, we had to consider and study so many factors. Those factors which we considered and how those factors affected for the inference process will be described below.

5.1.3.1.1 Architecture of the model

A model consists of different types of layers such as convolution layers, pooling layers and dense layers. We had to consider the number of convolution layers, number of pooling layers, kernel sizes, used stride and padding types to divide the inference process between devices. Those details were taken by traversing through the model architecture file.

5.1.3.1.2 Input partitioning method

We have to consider the most suitable way to divide the input within the resource-constrained devices. By input, we have to consider not only the initial input, but also the inputs to the middle layers such as convolution layers and pooling layers. If we consider that the initial input data is an Image, it can be two dimensional (black & white) or three dimensional (RGB). In our evaluated scenario, the initial input is an image which is three dimensional (x, y, z) where value 'x' is the length of image, value 'y' is the width of the image and value z is always equal to 3 referring to the red, green and blue channels of the image.

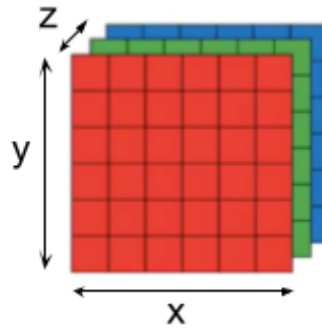


Figure 6: Initial Input Format

But the input data changes from layer to layer. So for a middle layer input data can be a large 3d matrix which has different x, y, z values depending on the number of kernels, kernel size, and stride values used in the convolution and pooling operations.

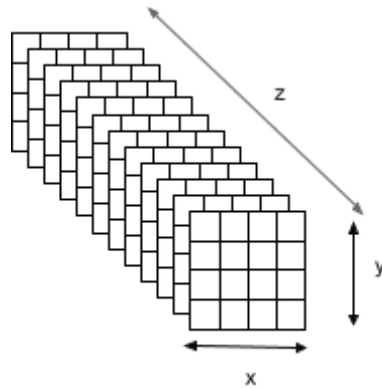


Figure 7: Common Input Format for any layer

If we consider, one method is to divide the input at any layer, through the length(x) or width(y). Another method is to divide the input through the matrix depth(z) between devices and do the processing. But in that method, there is a large overhead in communication between the Raspberry

Pi devices. That's because after each convolution layer, slave devices have to send their output data to the master device and merge them to take the next 3d matrix data input which is needed for next operations. In that method, both input data for layers and kernel data used for convolution should be divided accordingly which increases communication delay and overhead. Because of that, what we used is the first method. In that method, each layer input 3d matrix was divided through the width(y) using an algorithm developed by us, which will be explained in the next section.

5.1.3.1.3 Overlapping Data Frame Calculation

When we were using the above mentioned input wise partitioning method, we had to face some difficulties. One of the main problems was how to divide the input data between the available devices. If there are “n” idle devices which can be used for processing, we can't just divide the input data into ‘n’ parts and process. As an example if we have three idle devices, we can't just divide the input into equal three pieces through the width like below.

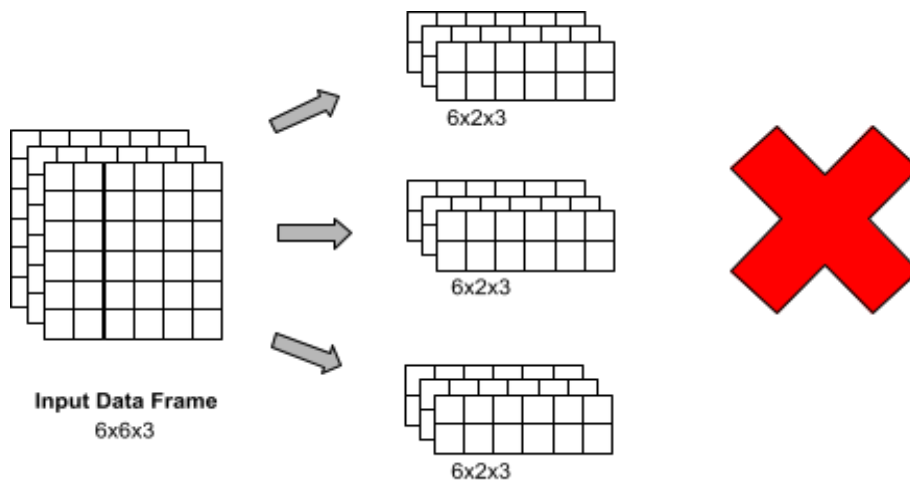


Figure 8: Not Suitable Input Partitioning Method

That's because if the input is equally divided, in the future processing, it will lose some required data for calculations. In that way we won't be able to get the accurate data at the end of the inference process. Because of that we have to do some overlapping before partitioning the input data.

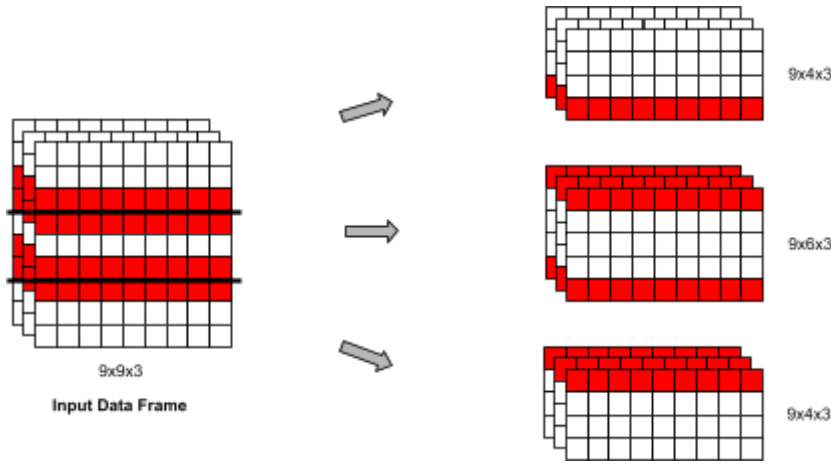


Figure 9: Input Partitioning with Overlapping

As an example, in the above input, without dividing the input by the black two thick lines, we do some overlapping with nearest other partitions, before partitioning. In the above picture red colored areas represent the areas used for overlapping. But that value of rows to overlap, cannot be decided by mind. As that value depends on the kernel sizes, stride values, no of convolution layers, no of pooling layers used for future processing, we had to find an algorithm that determines how many rows should be overlapped from nearest other partitions before partitioning. That algorithm will be explained below.

5.1.3.1.4 Overlapping Pixel Calculation Algorithm

Let's get the model layer architect is as follows

$$C_{11}C_{12}... C_{1a} P_1 C_{21}C_{22}... C_{2b} P_2 C_{31}C_{32}... C_{3c} P_c C_{41}C_{42}... C_{4d}....$$

Where,

- C_{ij} - Convolution layer x
- S_{ij} - Stride size for convolution layer x
- K_{ij} - kernel size for convolution layer x
- P_i - Pooling layer x
- S_{ip} - Stride size for Pooling layer x

$$\begin{aligned}
& \sum_{j=1}^a \left(\frac{K_{1j}-1}{2} \right) \left(\frac{S_{1j}}{1} \right) + \left(\sum_{j=1}^a \left(\frac{K_{2j}-1}{2} \right) \left(\frac{S_{2j}}{1} \right) \left(\frac{S_{1p}}{1} \right) \right) * \left(\max \left(\frac{K_{1j}-1}{2} \right) \left(\frac{S_{1j}}{1} \right) \right) + \\
& + \left(\sum_{j=1}^b \left(\frac{K_{3j}-1}{2} \right) \left(\frac{S_{3j}}{1} \right) \left(\frac{S_{1p}*S_{2p}}{1} \right) \right) * \left(\max \left(\frac{K_{1j}-1}{2} \right) \left(\frac{S_{1j}}{1} \right) \right) * \left(\max \left(\frac{K_{2j}-1}{2} \right) \left(\frac{S_{2j}}{1} \right) \right) + \\
& + \left(\sum_{j=1}^c \left(\frac{K_{4j}-1}{2} \right) \left(\frac{S_{4j}}{1} \right) \left(\frac{S_{1p}*S_{2p}*S_{3p}}{1} \right) \right) * \left(\max \left(\frac{K_{1j}-1}{2} \right) \left(\frac{S_{1j}}{1} \right) \right) * \left(\max \left(\frac{K_{2j}-1}{2} \right) \left(\frac{S_{2j}}{1} \right) \right) * \left(\max \left(\frac{K_{3j}-1}{2} \right) \left(\frac{S_{3j}}{1} \right) \right) + \\
& + \dots
\end{aligned}$$

Likewise, this addition should happen for all convolutional layers. This algorithm is valid for padding valid/ same situations.

5.1.3.2 Kernel Wise Partitioning (KWP)

In each convolutional layer we need to convolute the input data frame or input data volume with a set of kernels (filters). Time required to complete the convolution operation depends on the input data dimension and the number of kernels. The following diagram (Diagram number) shows a typical convolutional operation on given input data and the result of the convolutional operation.

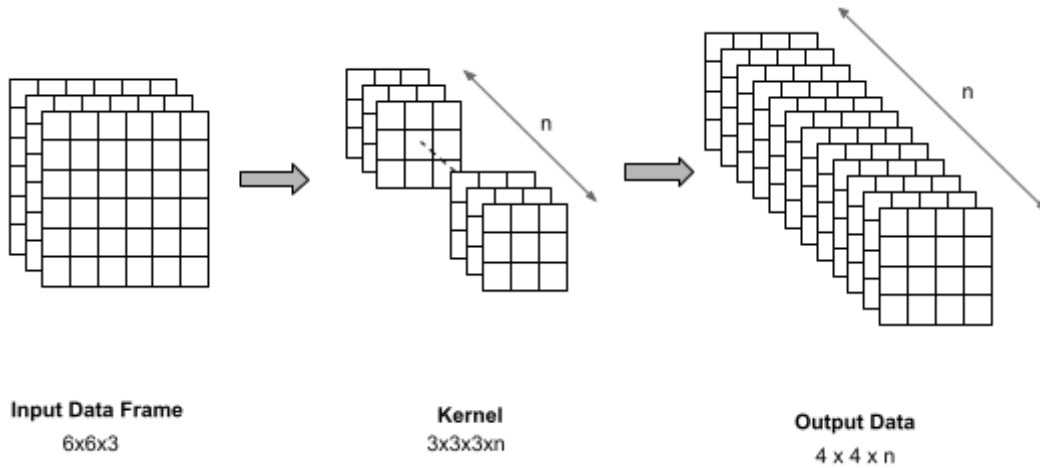


Figure 10: Convolutional operation with Kernel

Above figure shows the valid padding situation where it does not apply padding around the input data. Therefore the output data frame has shrunk in height and width.

The **6x6x3** dimension input data frame has been convoluted with *a number* of **3x3x3** kernels resulting in a **4x4xn** dimensional data matrix.

If the above convolutional operation happens on a single device. Depending on the value of ***n***, the convolution operation can take a substantial amount of time and memory to process the calculation.

In our Kernel Wise Partitioning (KWP) methodology we propose to break the 3x3x3xn kernels into ***r*** number of small partitions where as

$$\sum_{i=1}^r 3 \times 3 \times 3 \times x_i = 3 \times 3 \times 3 \times n$$

Where

$$\sum_{i=1}^r x_i = n$$

Assuming we have ***r*** number of devices we can separately and parallelly execute ***r*** number of partitioned convolution operations on each device resulting in large time and memory saving.

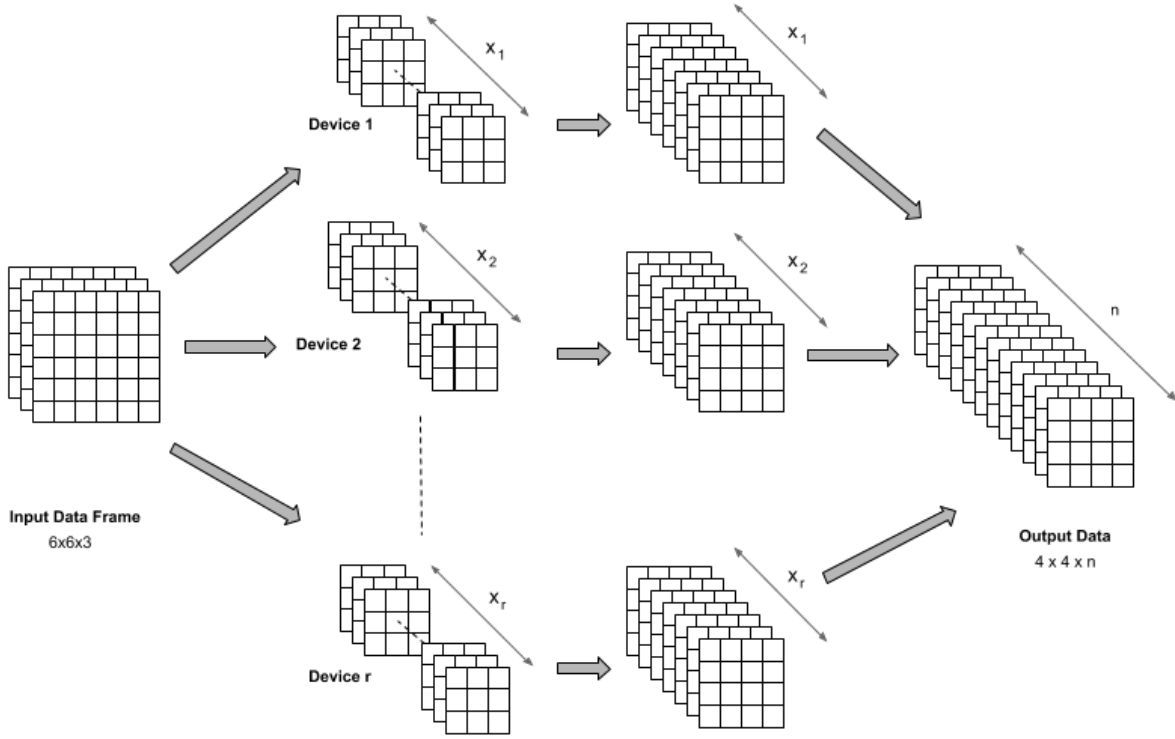


Figure 11: KWP in r number of devices.

Here we try to assign a set of kernels for each slave device and evaluate the time reduction. However, in this case, we need to send the entire input data to all slave devices which may increase the communication delay.

5.2 Implementation

In the second phase of this research we tried to implement different proposed partitioning schemas and evaluate their performance changes.

5.2.1 High Level System Design

Based on the above plan we designed the system architecture of the proposed framework.

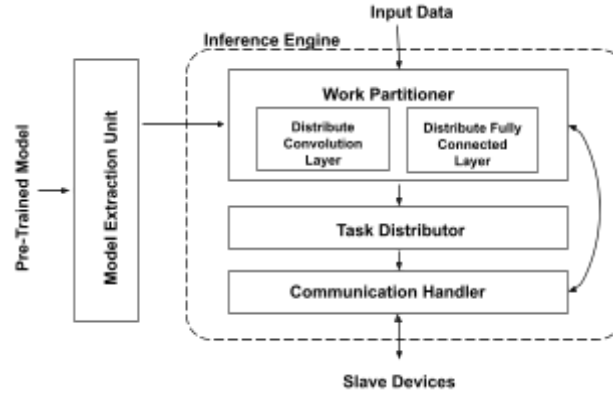


Figure 12: System Design of the DeepEdge Tool

5.2.2 Model Extraction Unit (MEU)

MEU is used to convert external, pre-trained models into single file format. We had to define which models that this tool supports and what kind of layers are supported. These constraints are defined based on the implementation of the distributed inference engine. Currently our MEU supports the YOLO v3 model.

MEU is designed to convert pre-trained models into a more optimized format which will be suitable for our distributed inference engine. In other words, conversion can be done in a way that will make it easy to partition the model according to our distribution scheme. Also pre-converting the models helps to reduce the execution time of the framework at runtime as we don't have to handle various types of formats at runtime.

According to our assumption each computational device needs to be set up before using it. Therefore once setup is completed each device will keep the converted model topology and weight data in separate files created by MEU. Therefore IE just needs to directly read those files at runtime without wasting time to convert each time it runs.

5.2.3 Inference Engine (IE)

Distribution of work will take place under two categories: Static distribution and Dynamic distribution.

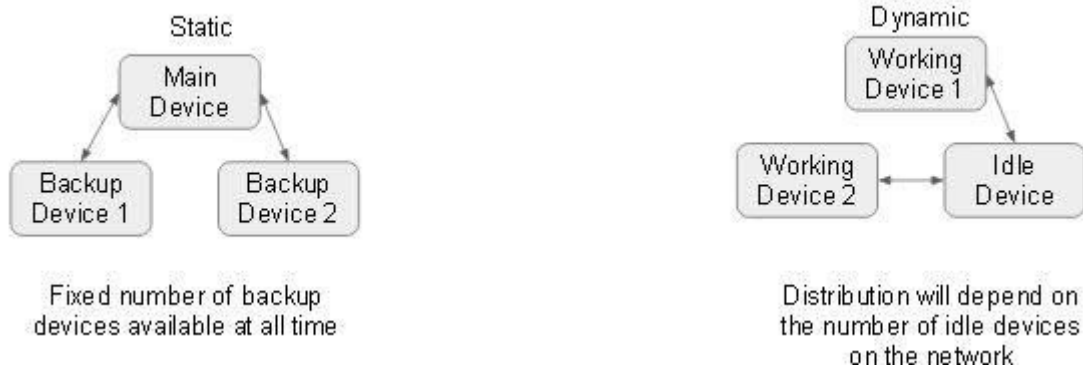


Figure 13: Task Distribution methods

Based on the type of the setup, the Model partition will decide how to allocate works to each and every available device.

In static the distribution, There is a fixed number of backup devices to support the main device which takes the input. In such a setup work will be partitioned equally among available devices.

In a dynamic setup, every device will be considered as an input taking device. In the initial setup and networking, we will define the number of devices in the network, So each device knows the maximum number of devices that can become ideal at a given time. Every device will partition the work based on the maximum number of devices in the network and prioritize those tasks considering the order of execution if a single device has to compute the task by putting them into a queue. Then the main device will start working from the highest priority task. Communication handler we proposed in section 4.2.2.1 above will be able to handle dynamic connections and broadcast tasks with the network of devices. If any of the devices is in the idle mode it will communicate with the main device to take the next priority task.

5.2.4 Communication demands

According to our assumption each device is pre configured. Therefore each device has the model topology and weight data. Therefore there is no need to transfer weight data or model data among devices at runtime.

However we will need to communicate the input data and output results. And instruction data which will indicate which part of the network needs to be calculated by each device.

Other than that various other system related communications will happen.

5.2.5 MEU Support for ONNX

As introduced before, ONNX (Open Neural Network Exchange) is a way to convert any model type into “onnx” format. As this can be used as an intermediate to convert models, we used an onnx type model as our next model type , Even Though we can convert other types to this format and use it for inference, onnx will become a major dependency of our tool.

We checked whether we can take the weights and architecture details out from an ONNX type model as those data can be used for distribution purposes at inference.

5.2.5.1 Alexnet

For testing purposes we used a pre-trained ONNX model with Alexnet CNN architecture. This neural network has 60 million parameters and 650,000 neurons, consists of five convolutional layers, some of which are followed by max-pooling layers, and three fully-connected layers with a final 1000-way softmax.

5.2.5.2 Alexnet architecture

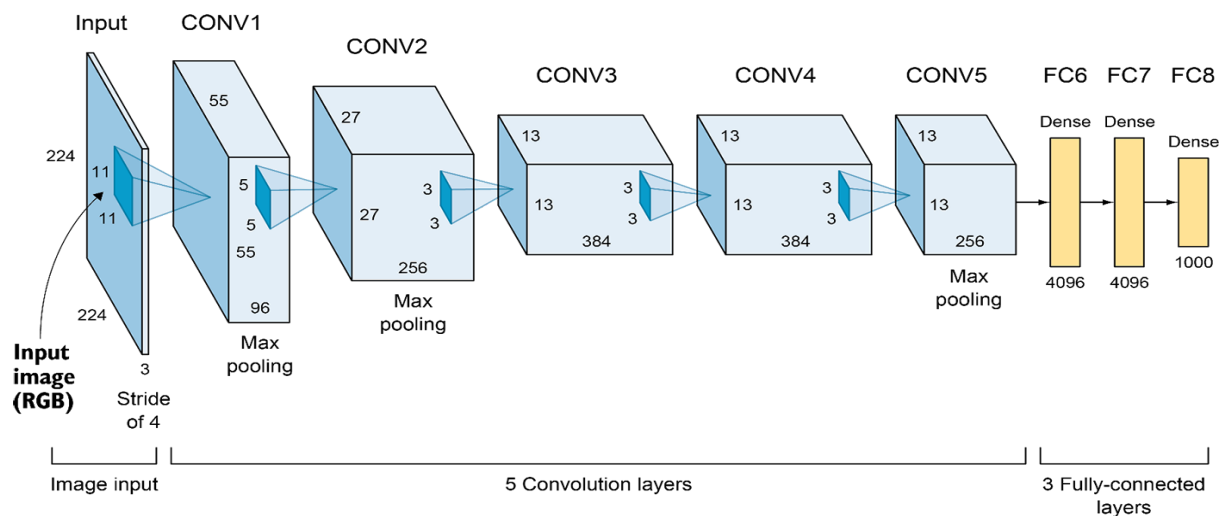


Figure 14: Alexnet Architecture (<https://towardsdatascience.com/>)

5.2.5.3 Steps for reading data inside model

- Model is first loaded using python language.
 - `model = onnx.load("model.onnx")`
- Weight values are read, by iterating through “model.graph.initializer” which consists of tensor proto type objects which includes the data at each node.
- Weight values at each node, which are encoded as “raw_data” should be converted to numeric values with the help of “onnx.numpy_helper”.

5.2.6 Building a Raspberry Pi Cluster

To make a Raspberry Pi cluster, we used four raspberry pis which are the model 3 B + of the raspberry pi series. Those raspberry pis have the following specifications

- Broadcom BCM2837B0, Cortex-A53 (ARMv8) 64-bit SoC @ 1.4GHz
- 1GB LPDDR2 SDRAM
- 2.4GHz and 5GHz IEEE 802.11.b/g/n/ac wireless LAN, Bluetooth 4.2, BLE
- Gigabit Ethernet over USB 2.0 (max 300 Mbps).

For the data communication switch we used a router which has 4 ethernet ports with WIFI connection to the remote Laptop/PC. Raspberry Pi to Raspberry Pi communication (master to slave/slave to master) happens via a switch. Specification of the router is as follows.

- TP LINK TD-W8901G
- LAN : IEEE 802.3u -100 Mbps
- WLAN : IEEE 802.11 b/g - 24Mbps

When we consider the distributed systems, one of the main factors that affects the time taken to complete a task is the communication delay.

Following experiment was done to demonstrate the effect of the communication delay. In this experiment we send image files to the slave nodes for the calculations. Firstly, image data was

converted to a string using the base64 encoding. Then the encoded string was sent to the slave device. Below diagram shows the relationship between communication delay vs image size.

5.3 Inference Task Distribution

5.3.1 Method 1

The First method we tried is to run everything on the main thread of the server device which triggers the distributed model execution. This method ensures that we won't run into issues like waits, locks and difficulty to identify errors. In this method everything happens in a synchronized manner. However once partitioned data is sent to a secondary device main device will immediately start executing the respective part of the task. Therefore only the communication happens in a synchronized way. Distributed processing happens parrarally in multiple devices.

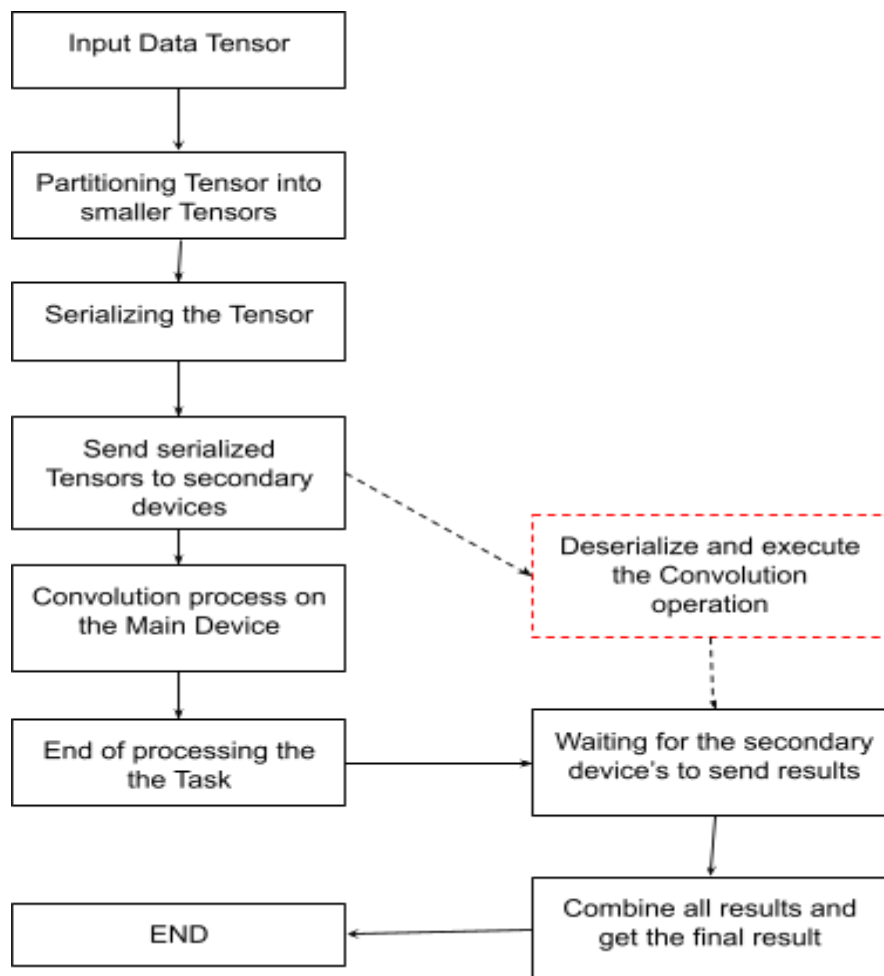


Figure 15: Single Threaded IE

The time consumption for convoluting different sizes of images were calculated and recorded. Test was done three times and the average value was taken.

5.3.2 Method 2

In method 2, we used threads to speed up the process. We identify some operations that can execute parallely and independently. Here we execute several processes in a secondary thread other than the main thread.

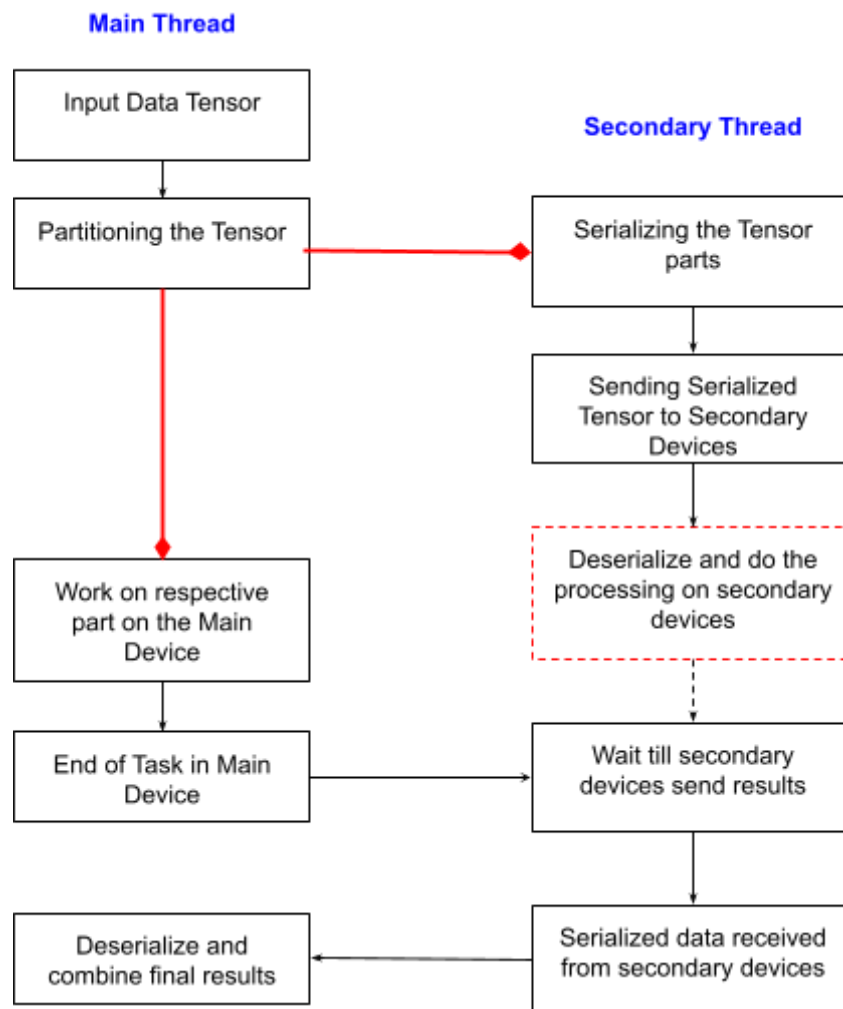


Figure 16: Distributed multi Thread Convolutional operation flowchart

Both method 1 and method 2 do the same work. Only difference is the use of threads in the second method. We can notice slight improvement of process speed in the second method as shown in the below chart.

5.3.3 Comparison between method 1 and method 2

Let's do the comparison between two methods. If we plot all the diagrams in one chart, (distributed system with and without the threading method) we can not clearly see the difference between those two lines (light blue and yellow lines).

Time (s) vs Image Size

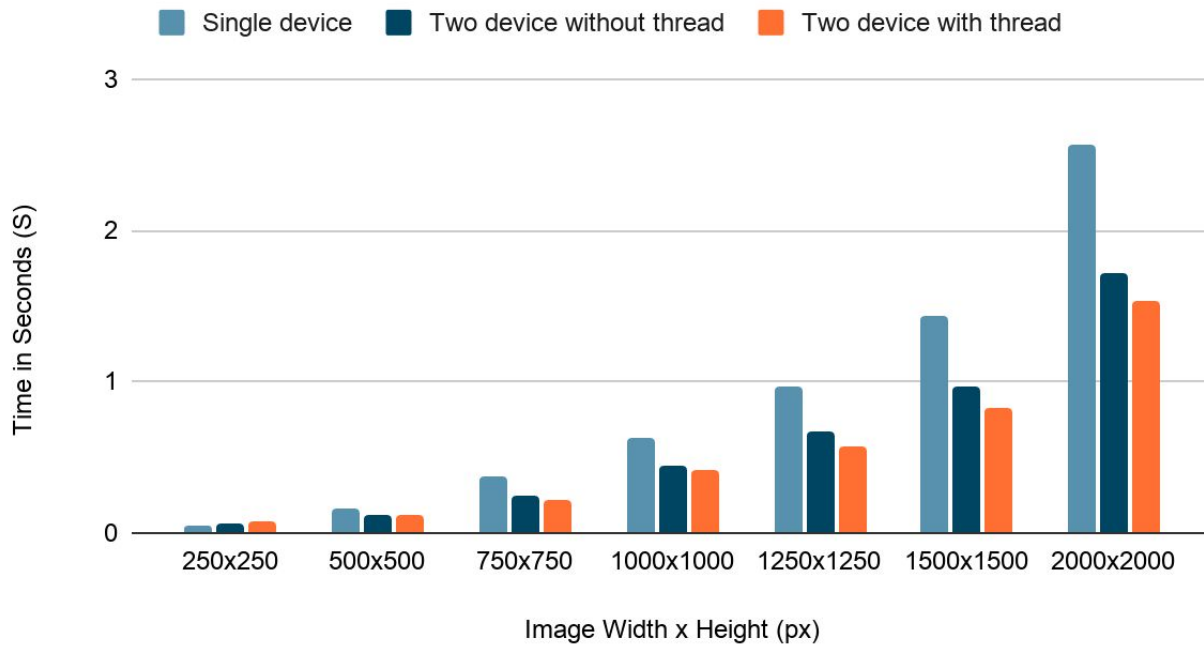


Figure 17: Image resolution vs time consumption single thread and multi thread comparison

However if we zoom into more, at the beginning of 250x250 point, method 2 consumes more time than other methods. However when it comes to the 500x500 point, method 1 and method 2 consume the same time. After passing that point method 2 showing better performance. Therefore In the range of 500x500 - 2000x2000 method 2 shows better performance compared to method 1. According to the experimental results, method 2 shows the best results in higher resolution images.

6 Evaluation

Evaluation process and the results obtained will be discussed In this section. We were able to evaluate the proposed partitioning schema considering various constraints that can affect the results. According to the evaluation results we were able to obtain significant improvements in inference time using distributed execution of CNN/DNN models.

6.1 Evaluation on a Custom Model

6.1.1 Model Architecture Introduction

For the evaluating purposes of the distributed inference process, we used mainly two model types. First model is a custom built tensorflow model that has trained to classify input data into five class types. Figure 18 shows the structure of the model that was used as the custom model. This model consists of 2 convolutional layers followed by two pooling layers and a dense layer at the end. Training was done on a separate machine and used the trained model for inference inside Raspberry Pi devices.

Layer (type)	Output Shape	Param #
rescaling_4 (Rescaling)	(None, 300, 300, 3)	0
conv2d_6 (Conv2D)	(None, 300, 300, 64)	1792
max_pooling2d_6 (MaxPooling2)	(None, 150, 150, 64)	0
conv2d_7 (Conv2D)	(None, 150, 150, 128)	73856
max_pooling2d_7 (MaxPooling2)	(None, 75, 75, 128)	0
flatten_2 (Flatten)	(None, 720000)	0
dense_2 (Dense)	(None, 5)	3600005

Figure 18: Custom Tensorflow Model Architecture

As described in the above figure, this model consists of two convolution layers, two pooling layers and one dense layer. At first the model rescales the input into the size that it needs for processing. In the above case, that required input is a 300 x 300 x 3 sized 3D tensor. But we trained models for different input sizes also such as 200px, 250px, 300px, 350px and 400px. First what we did was, dividing the inference process between two Raspberry Pi devices. One device will act as a main device, which will initiate the process. It will send a calculated amount from input data (considering the architecture of the model), to the secondary device for the inference process, and will process the remaining part by itself. Secondary device will process the data it receives, and after processing it will send the result data to the main device. Main device will do the necessary merging to the data

and give the final output. We have calculated the average time taken for this distributed inference process between two devices and listed below. We did this distribution in both types, ethernet and wifi which were used as the communication medium between devices. For the custom model I did the evaluation for three and four devices.

6.1.2 Input size vs Inference time (Three Devices)

Next what we did was increase the resource constrained device count for distributing inference. So we tested for three raspberry pi devices. In there also, there is one master device which will initiate the process and two slave devices for distributed processing. Average time taken for the inference, with different input sizes for both communication methods (ethernet and wifi) are listed below.

Input Size	Single Device	Three Devices - Ethernet	Three Devices - Wifi
200px	1.079006523	0.387830585	0.5743546486
250px	1.397940606	0.543470532	0.8099578619
300px	1.585931093	0.633701086	1.044556226
350px	2.248090774	0.996847232	1.488068458
400px	2.78399694	1.352278292	1.909108997

Table 2: Average inference time for different input sizes (using three devices)

Inference Time vs Input Size

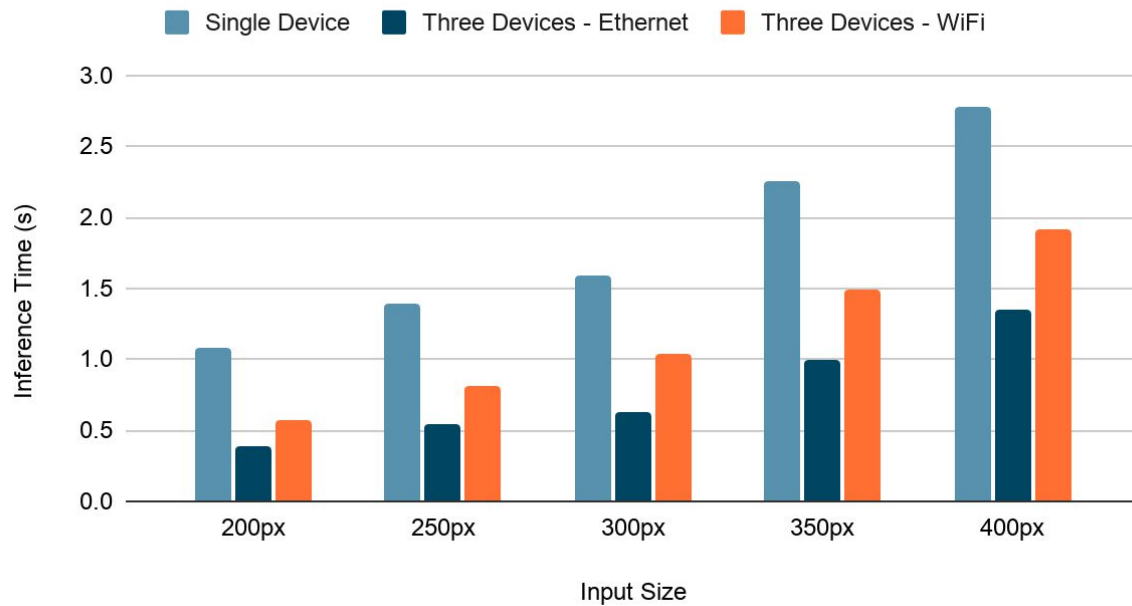


Figure 19: Average inference time for different input sizes (using three devices)

As you can see, there is a speedup when it comes to distributing the inference between three devices too compared to a one device, which is listed below.

Input Size	Ethernet - speedup	Wifi - speedup
200px	64.05 %	46.77 %
250px	61.12 %	43.06 %
300px	60.04 %	34.14 %
350px	55.65 %	33.80 %
400px	51.42 %	31.43 %

Table 3: Average inference speedup gain for different input sizes (using three devices)

Even though when using ethernet, time for inference is slightly reduced with the increment of input size, there are some ups and downs for the speedup gained when using wifi with the increment of input data size. But for both communication methods, there is a speedup gain when compared to the inference at one device.

6.1.3 Input size vs Inference time (Four Devices)

Finally for the distributed inference for the custom model, we increased the device count used up to four. In this case, there is one master device which will initiate the process and three slave devices for distributed processing. Average time taken for the inference using four raspberry pi devices with different input sizes for both communication methods (ethernet and wifi) are listed below.

Input Size	Single Device	Four Devices - Ethernet
200px	1.079006523	0.377960265
250px	1.397940606	0.471646041
300px	1.585931093	0.60384798
350px	2.248090774	0.871758223
400px	2.78399694	1.074187577

Table 4: Average inference time for different input sizes (using four devices)

Inference Time vs Input Size

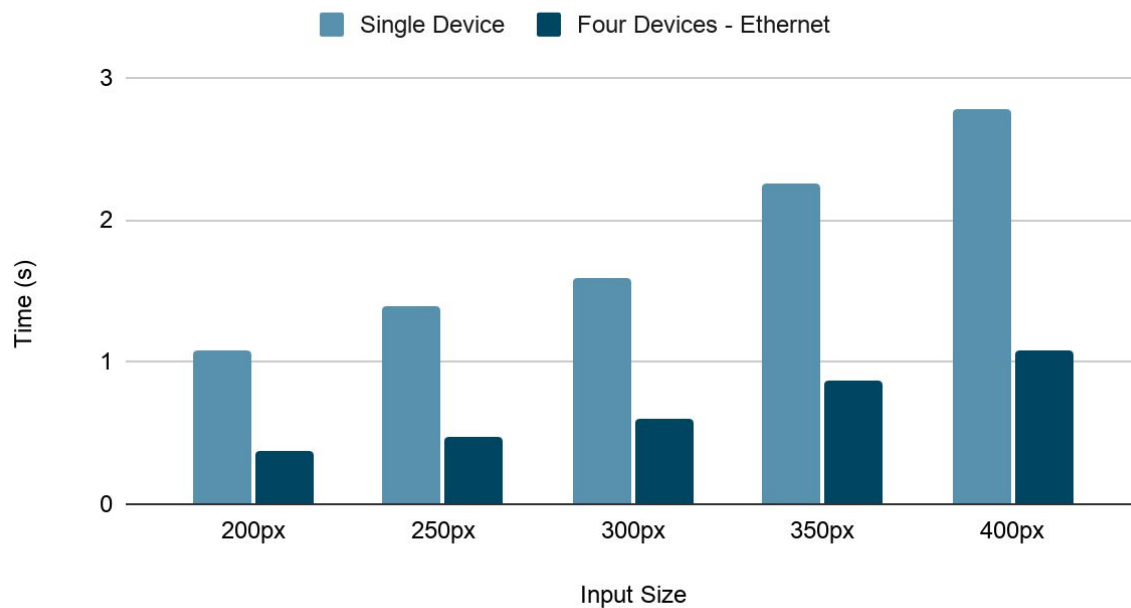


Figure 20: Average inference time for different input sizes (using four devices)

As you can see, there is a speedup when it comes to distributing the inference between four devices too compared to a one device, which is listed below.

Input Size	Ethernet - speedup
200px	64.97 %
250px	66.26 %
300px	61.92 %
350px	61.22 %
400px	61.41 %

Table 5: Average inference speedup gain for different input sizes (using four devices)

From the above results for distributed inference in four devices, we can't identify any significant change in speedup gain with the increment of input size, when ethernet is used as the communication method. The main reason for the large percentage of speedup is we distribute the fully connected layer also. Due to that, slave devices only need to send a small amount of data (5 values) to the main device.

We were unable to test the performance of four devices with WiFi communication as there were only 4 Raspberry Pi devices and one of them had an issue with the WiFi connection. Testing on WiFi connection gave strange results due to this issue. Therefore we removed the affected results from the evaluation.

6.1.4 Device count vs Inference Time (Ethernet)

Now let's compare what happened for the average time taken for a certain input data size, with the increment of used device count for distributing the inference process. Following chart shows how that average time was changed, when the ethernet was used as the communication method.

Inference Time (Ethernet) vs Device Count

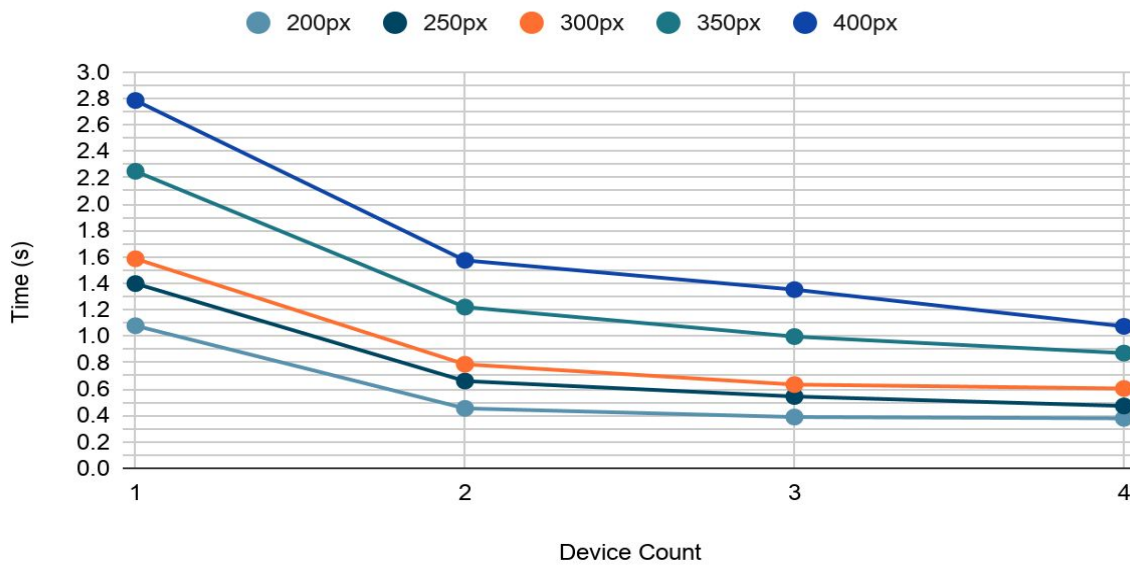


Figure 21: Average inference time change with number of devices used (ethernet medium)

As you can see, whatever the input size is, average time taken for the inference process is decreased with the increment of the number of devices used for the distributed processing. But with the increment of the number of devices, the magnitude of the decrease in average time for inference is reduced. When studying the above chart we can see that, between three and four device counts, there isn't much difference in average times recorded, especially for low input data sizes. So probably, using more than four devices (n devices) especially for low input data sizes, won't have much benefit, because speedup gain compared to using $(n-1)$ devices will be very low. Because of that depending on the scenario, we have to find the number of devices, which will give a considerable speedup also considering the cost for a device.

6.1.5 Device count vs Inference Time (WIFI)

Let's compare what happened for the average time taken for a certain input data size, with the increment of used device count for distributing the inference process when wifi was used as the communication method between devices.

Inference Time (Wifi) vs Device Count

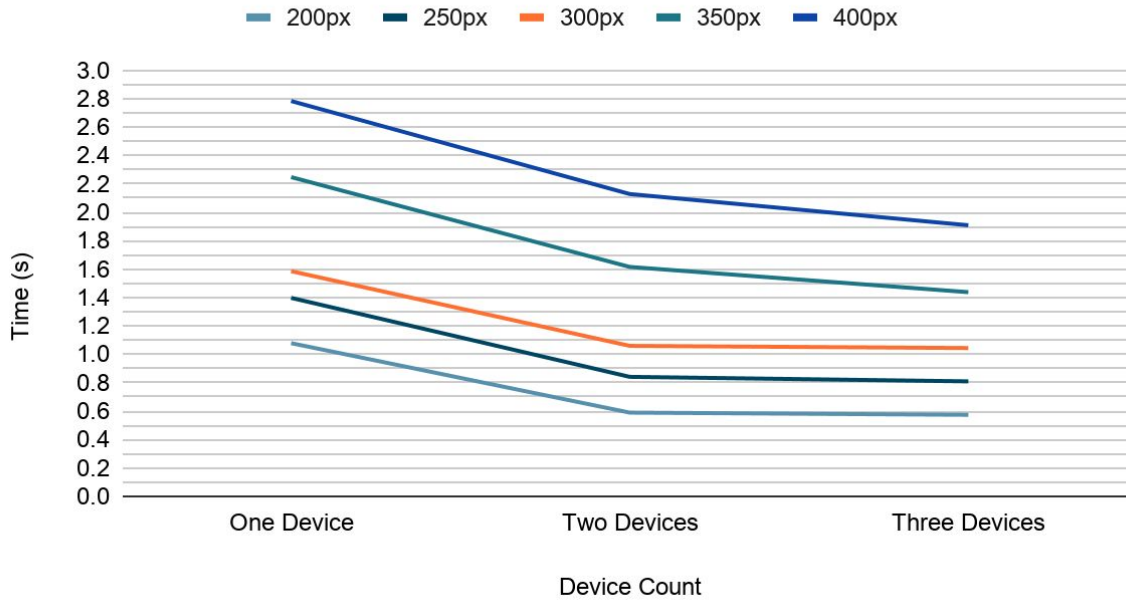


Figure 22: Average inference time change with number of devices used (wifi medium)

Using the above chart we can't come to a generic conclusion like we did for the chart when the ethernet was used as the communication. That's because some ups and downs of values of average time were observed, when increasing the number of devices used. But it was observed that, rather than doing the inference process in one device, there is a speedup gain of inference when distributed processing is done using 2 or more devices. That conclusion is true, whether the communication method is ethernet or wifi.

6.2 Evaluation on VGG 16 Model

6.2.1 Model Introduction

This is the second type of model that we used for evaluation. Here we were able to test our proposed algorithm with a standard CNN architecture due to the fact that we used the VGG16 model which is widely used in computer vision. VGG16 is a convolutional neural network model proposed by K. Simonyan and A. Zisserman from the University of Oxford in the paper "Very Deep Convolutional Networks for Large-Scale Image Recognition". This model achieves 92.7% top-5 test accuracy in ImageNet, which is a dataset of over 14 million images belonging to 1000

classes. It was one of the famous models submitted to [ILSVRC-2014](#). It makes the improvement over AlexNet by replacing large kernel-sized filters (11 and 5 in the first and second convolutional layer, respectively) with multiple 3×3 kernel-sized filters one after another. Architecture of the model is described in the below image.

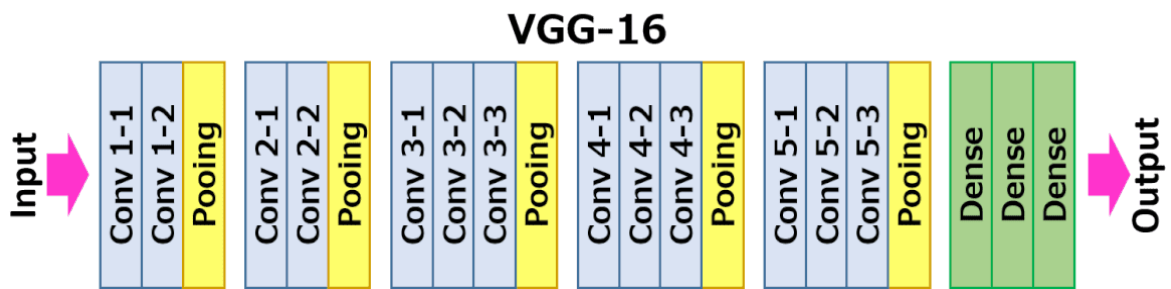


Figure 23: VGG 16 Model Architecture

However the main architecture is defined for 1000 classes at the final dense layer. The graphical views of the model is given in the below figure.

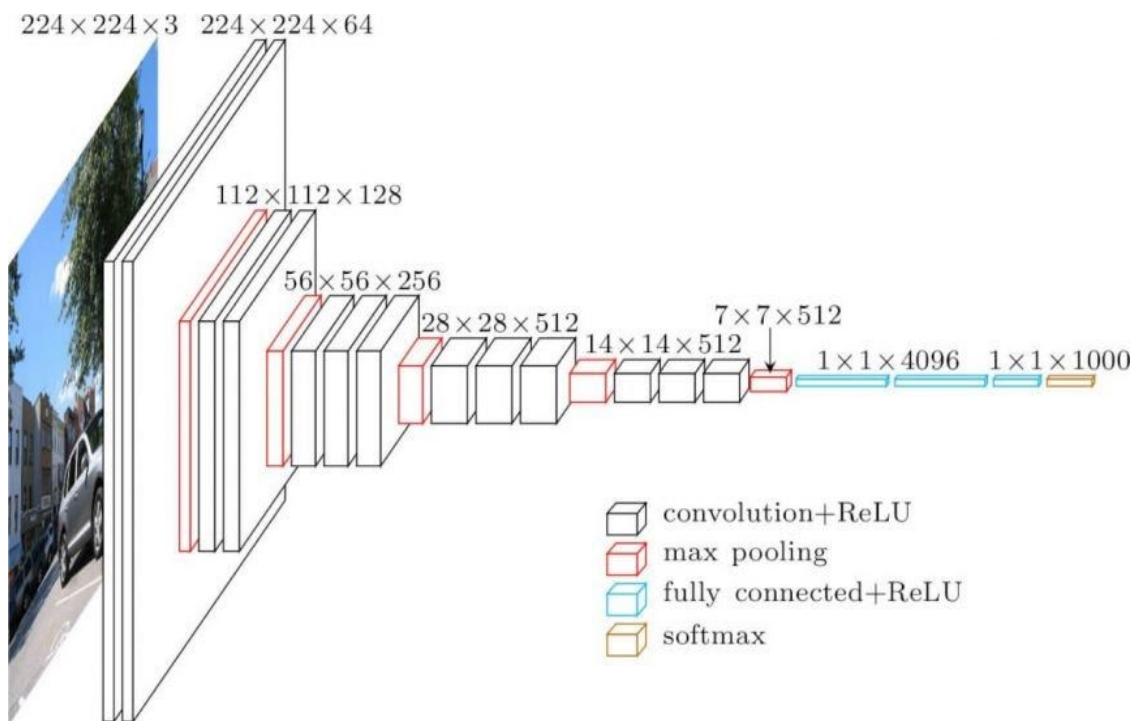


Figure 24: VGG 16 Model Architecture Graphical View

In our case we tested the VGG16 model with custom trained five classes which mean now the final dense layer will have five neurons at the end. Also we changed the neurons amount in the first two dense layers because having 4096 neurons in those dense layers is too big for a small amount of

classes. In the standard VGG16 model it has 1000 different classes. When it goes from flatten layer to first dense layer so the calculation happens with more than 10 Millions parameters. In our case we reduced that amount to about 300 hundred thousand parameters.

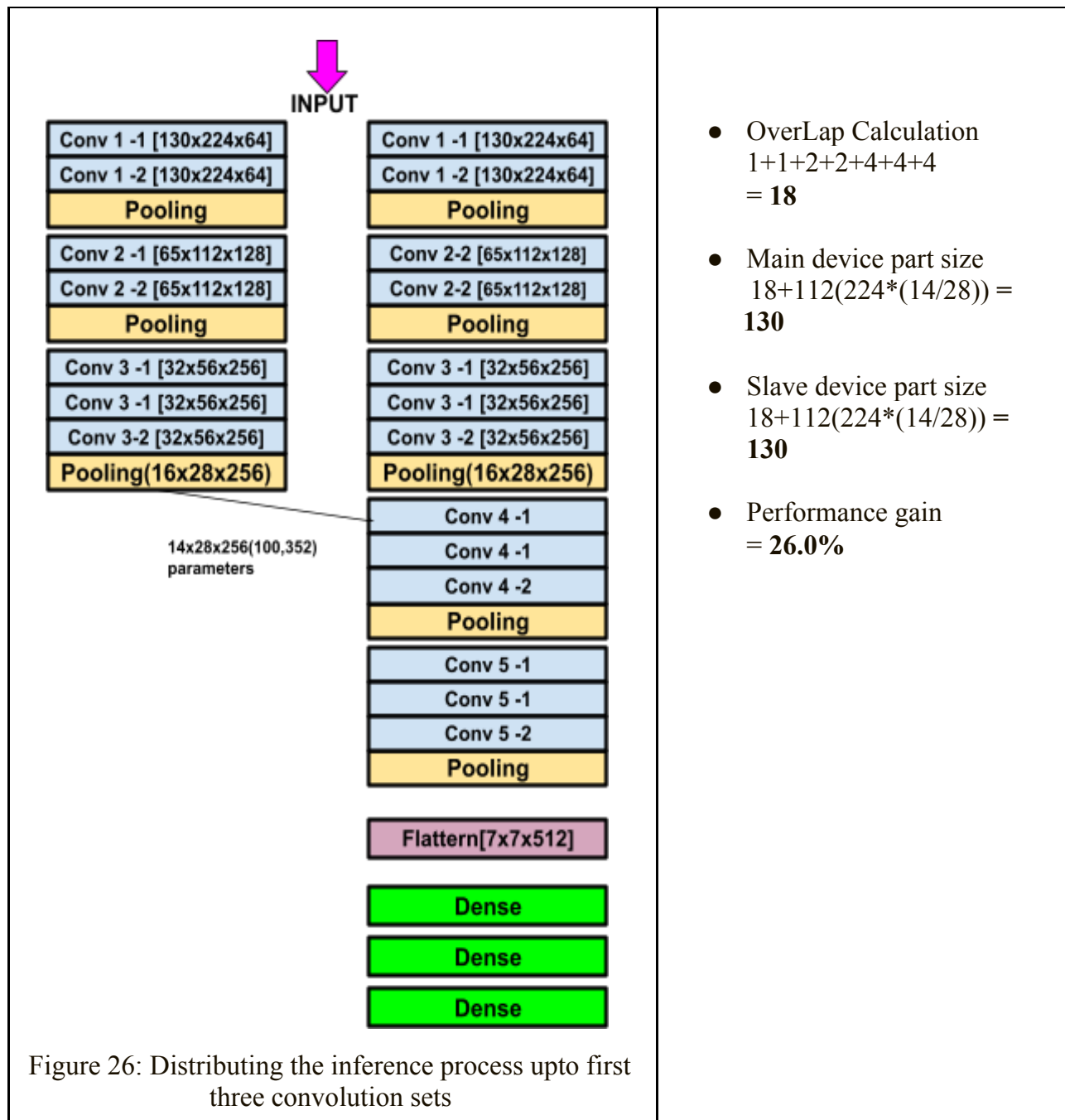
Layer (type)	Output Shape	Param #
block1_conv1 (Conv2D)	(None, 224, 224, 64)	1792
block1_conv2 (Conv2D)	(None, 224, 224, 64)	36928
block1_pool (MaxPooling2D)	(None, 112, 112, 64)	0
block2_conv1 (Conv2D)	(None, 112, 112, 128)	73856
block2_conv2 (Conv2D)	(None, 112, 112, 128)	147584
block2_pool (MaxPooling2D)	(None, 56, 56, 128)	0
block3_conv1 (Conv2D)	(None, 56, 56, 256)	295168
block3_conv2 (Conv2D)	(None, 56, 56, 256)	590080
block3_conv3 (Conv2D)	(None, 56, 56, 256)	590080
block3_pool (MaxPooling2D)	(None, 28, 28, 256)	0
block4_conv1 (Conv2D)	(None, 28, 28, 512)	1180160
block4_conv2 (Conv2D)	(None, 28, 28, 512)	2359808
block4_conv3 (Conv2D)	(None, 28, 28, 512)	2359808
block4_pool (MaxPooling2D)	(None, 14, 14, 512)	0
block5_conv1 (Conv2D)	(None, 14, 14, 512)	2359808
block5_conv2 (Conv2D)	(None, 14, 14, 512)	2359808
block5_conv3 (Conv2D)	(None, 14, 14, 512)	2359808
block5_pool (MaxPooling2D)	(None, 7, 7, 512)	0
flatten (Flatten)	(None, 25088)	0
dense (Dense)	(None, 128)	3211392
dense_1 (Dense)	(None, 128)	16512
dense_2 (Dense)	(None, 5)	645

Changed only the fully connected layer architecture because having 4096 neurons in first two dense layers are too much for the small amount of calcification classes. Also 4096 neurons per dense layer makes the model size too big, because it has to store around 10 million weight parameters, required for calculations with those neurons.

Figure 25: Change done to VGG 16 Model Architecture

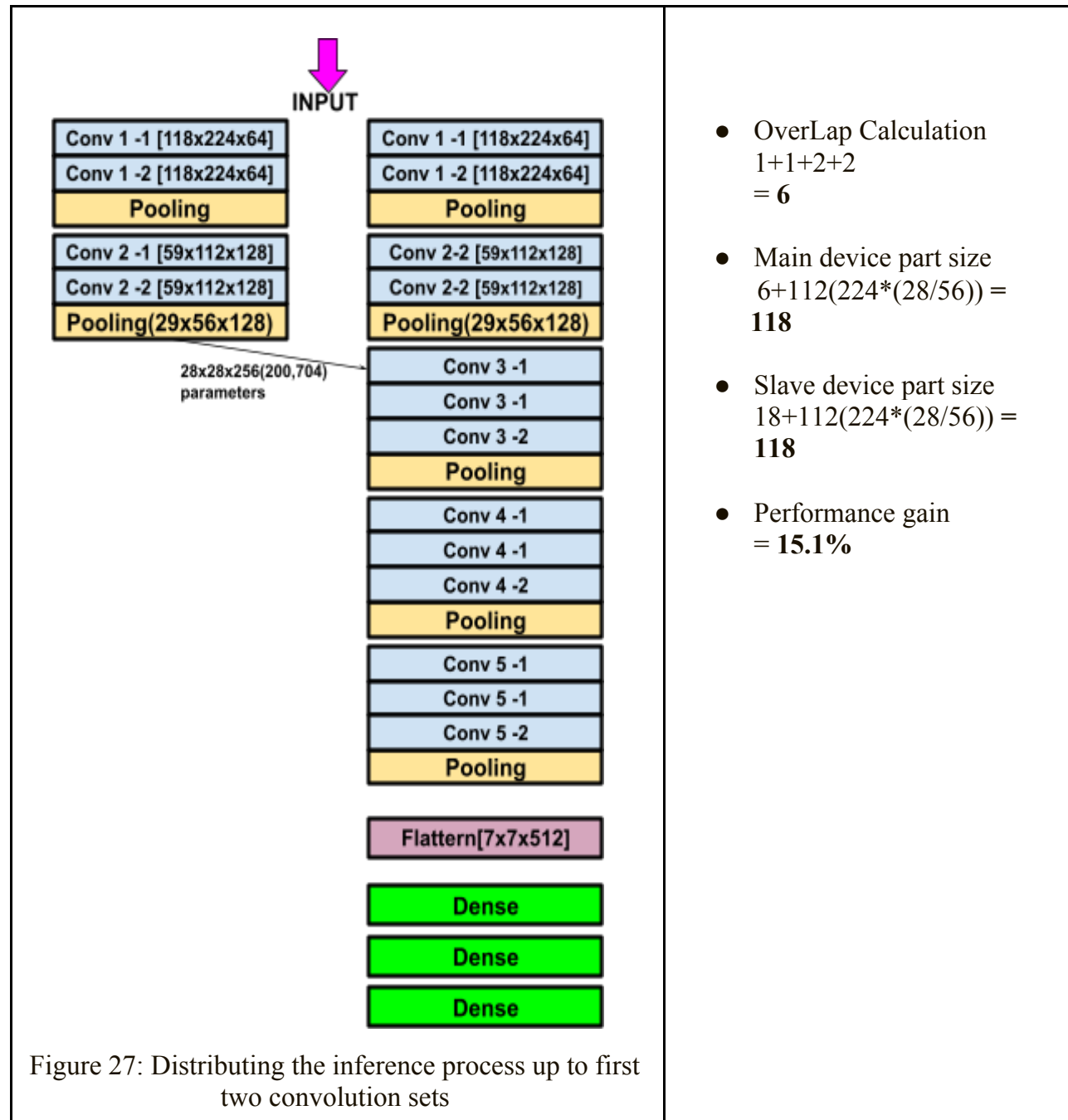
So we tried different kinds of partitioning methods for the VGG16 model which will be explained below. In those methods explained below, partitioning is done between two devices and given input and output parameters are defined in the following grids. In the previous evaluation method done

for the custom model, we distributed the whole inference process, between whatever the number of devices. But in this case, we had to do a large overlapping between partitions, in order to distribute the whole inference process. Following are the evaluations done by me for the VGG-16 model using two devices.



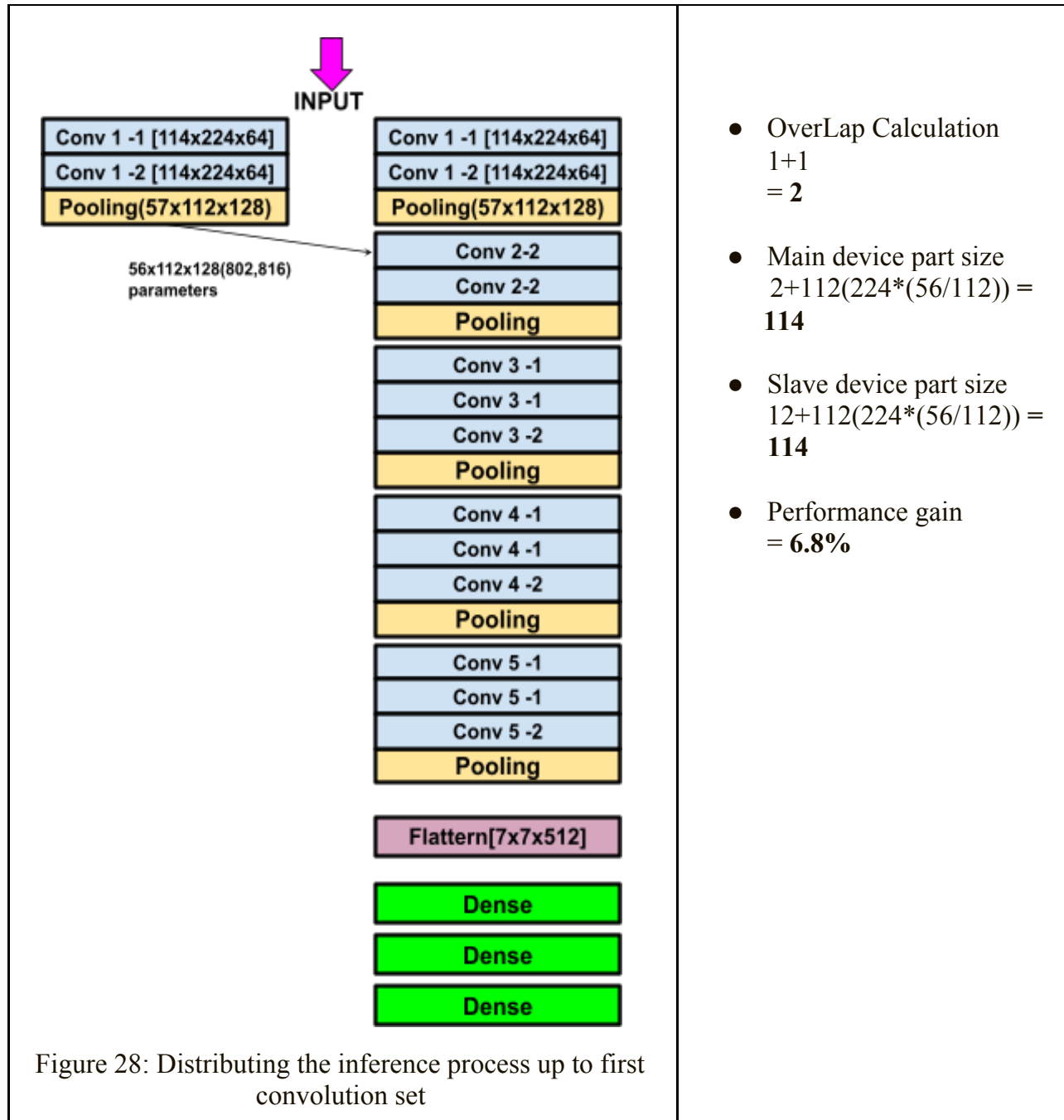
In this way, we reduce the convolution sets which are processed distributedly, upto 3. So the first three levels of convolution sets are done by distributed processing. As you can see, the overlapping amount of the partitions gets further reduced because we distribute the model up to Conv-3 level. But at the end of the processing of the slave device it has to send more then one hundred thousand

parameters to the main device which is larger in amount compared to the second method. So the communication delay is increased. Because of that compared to the second way, speedup gain is reduced by 2.8%.



In this fourth way, the convolution sets which are processed distributedly, were reduced upto 2. So the first two levels of convolution sets are done by distributed processing. As you can see, the overlapping amount of the partitions gets further reduced because we distribute the model up to Conv-2 level. But at the end of the processing of the slave device it has to send more parameters to the main device which is larger in amount compared to the third method. So the communication

delay is increased. Because of that compared to the third way, speedup gain is reduced by 8.9%.



In this last type of distributing between two devices, we tried distributing only the first convolution set. Even though the overlapped amount is further reduced, data size communicated between master and slave after the processing done by slave is larger than the previous types tested. Because of that performance is further reduced compared to the previous types. But still there is speedup gain compared to the non-distributed inference processing using only one device.

Results taken from all methods done by us are displayed in a chart below. We can see that maximum performance was observed, when doing distributed processing up to the first four convolution sets. Fifth convolution set and dense layers calculations should be done at the main device.

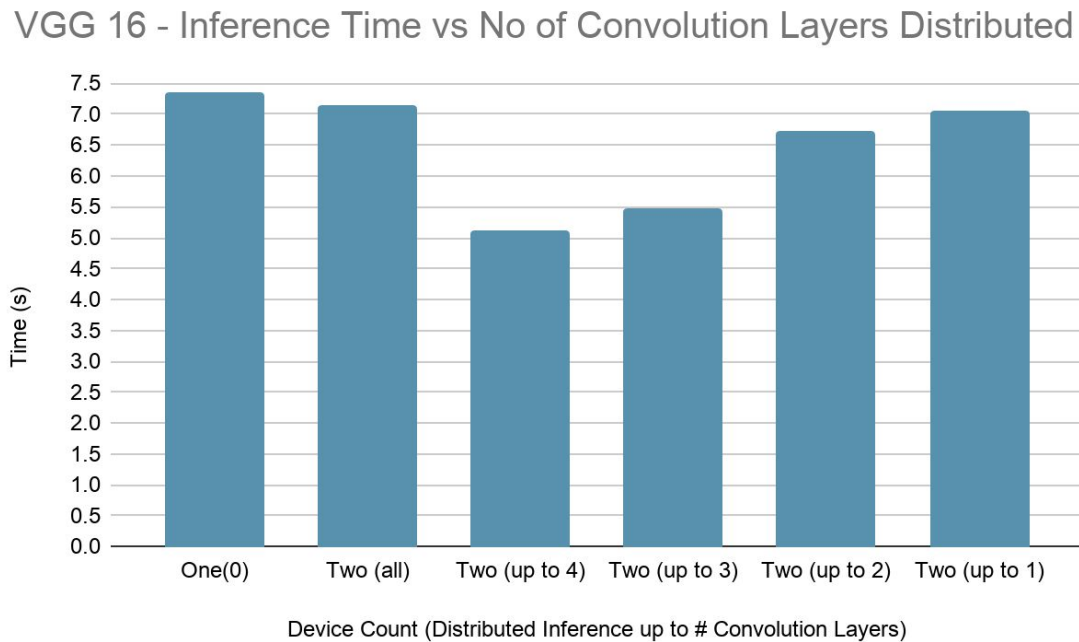


Figure 29: Average inference time change with the amount of distributed processing

In the above partitioning types, the communication method used was the ethernet. We did testing using wifi as a communication method also, for the type which was identified as giving the best performance with ethernet. So we did a distributed inference process up to 4 convolution sets, with wifi as a communication method between devices.

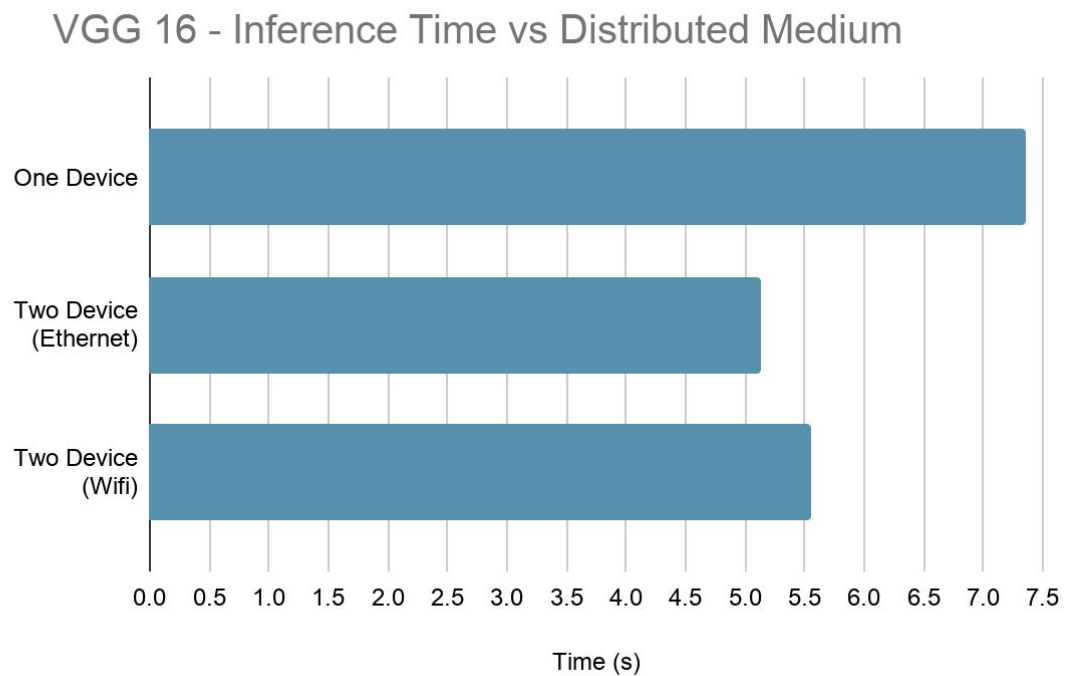


Figure 30: Average inference time for wifi and ethernet

From the chart above, we can conclude that even with wifi, for a standard model like VGG 16, there is a performance gain by distributing the inference, rather than doing the inference at one device. Another evaluation we did was increasing the number of devices upto 3 for distributing inference for the VGG 16 model. Following chart shows the average time taken for that, compared to using one or two devices.

VGG 16 Inference Time vs Used Device Count

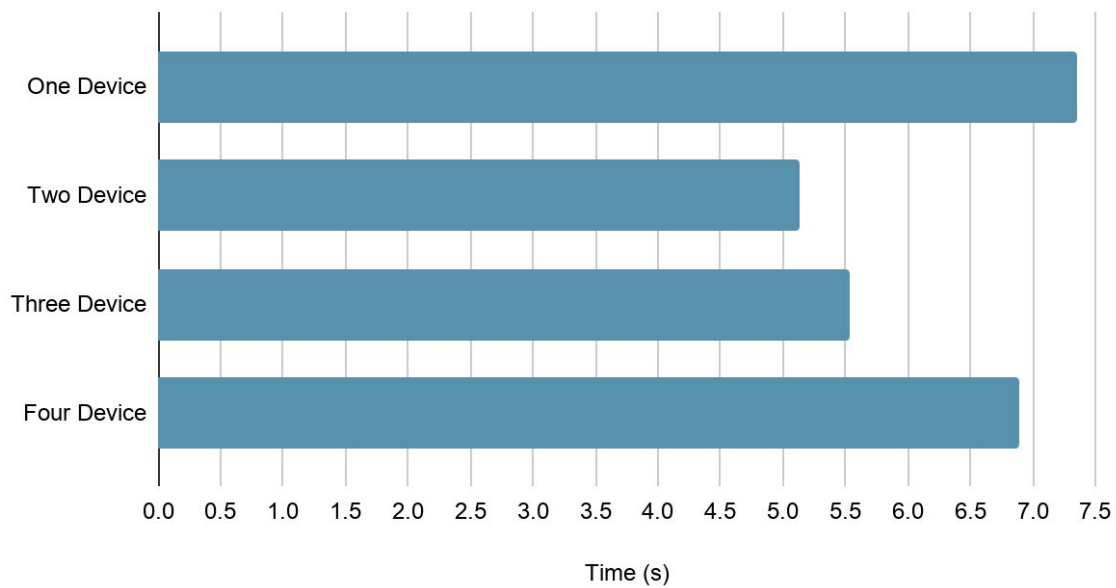


Figure 31: Average inference time vs used Device Count

Above results show that using three and four devices have a performance reduction compared to two devices. In this test, distributed processing was done only upto first four convolution sets, which gave the best performance when processed with two devices. Reason for the performance reduction is, there is a large overlap between partitions that we distribute and process. Because of that communication delay is increased. But still there is a speedup of inference compared to doing the whole inference process at one device.

7 Conclusion/Summary

In conclusion, this project was intended to propose and develop a novel approach that can be used to distributively execute the inference of convolutional neural networks. We were able to propose two partitioning schemes where as Input Wise PArtitioning (IWP) and Kernel Wise Partitioning (KWP) for convolutional layers and one fully connected layer partitioning schema. Also we proposed a task sharing and communication handling mechanism among devices. Furthermore, we implemented a program to evaluate the performance of proposed schemas using Python. Evaluation was done on a Raspberry Pi setup connected over the wired and wireless mediums.

From the evaluation it was found that the Input Wise Partitioning method reduces the time taken for inference tasks and makes it possible to run large neural networks which usually cannot be deployed to a single device. Furthermore, we were able to investigate the ideal method for partitioning to reduce communication demand among the Raspberry Pis to prevent unwanted communication overheads. However the Kernel Wise Partitioning method failed to give better results due to extensive network communication that was required to happen in each layer.

Using the proposed partitioning schema we were able to achieve nearly upto 60% inference speedup with ethernet communication medium and nearly upto 40% inference speedup with WiFi communication medium while preserving the accuracy level at 100%.

Further improvements to the proposed solution can be investigated by doing optimization on used algorithms, libraries, programming languages and network connections. Also we can investigate the use of fused partitioning schema for CNNs using both IWP and KWP methods. As most of the time input data dominates the initial layers of a CNN and kernels dominate in the later layers, we can apply IWP for early layers and KWP for later layers to evaluate the performance improvement that can be achieved. Furthermore we can evaluate the use of proposed methods on CPU/GPU clusters and the applicability of proposed solutions in the training phase can also be investigated.

References

- [1] J. Chen and X. Ran, “Deep Learning With Edge Computing: A Review,” *Proc. IEEE*, vol. 107, no. 8, 2019.
- [2] S. Liu, Y. Lin, Z. Zhou, K. Nan, H. Liu, and J. Du, “DeepIoT: Compressing deep neural network structures for sensing systems with a compressor-critic framework,” in *Proc. SenSys*, 2017, pp. 1–4.
- [3] X. Wang, X. Wang, and S. Mao, “RF sensing in the Internet of Things: A general deep learning framework,” *IEEE Commun. Mag.*, vol. 56, no. 9, pp. 62–67, Sep. 2018.
- [4] C. Liu et al., “A new deep learning-based food recognition system for dietary assessment on an edge computing service infrastructure,” *IEEE Trans. Services Comput.*, vol. 11, no. 2, pp. 249–261, Jan. 2018.
- [5] A. H. Jiang et al., “Mainstream: Dynamic stem-sharing for multi-tenant video processing,” in *Proc. USENIX Annu. Tech. Conf. (USENIXATC)*, 2018, pp. 29–42.
- [6] Y. Kang et al., “Neurosurgeon: Collaborative intelligence between the cloud and mobile edge,” *ACM SIGPLAN Notices*, vol. 52, no. 4, pp. 615–629, 2017.
- [7] J. Mao, X. Chen, K. W. Nixon, C. Krieger, and Y. Chen, “MoDNN: Local distributed mobile computing system for deep neural network,” in *Proc. Design, Autom. Test Eur. Conf. Exhibit. (DATE)*, Mar. 2017, pp. 1396–1401.
- [8] Z. Zhao, K. M. Barijough, and A. Gerstlauer, “Deepthings: Distributed adaptive deep learning inference on resource-constrained iot edge clusters,” *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 37, no. 11, pp. 2348–2359, Nov. 2018.
- [9] S. Teerapittayanon, B. McDanel, and H. Kung, “Distributed deep neural networks over the cloud, the edge and end devices,” in *Proc. IEEE 37th Int. Conf. Distrib. Comput. Syst. (ICDCS)*, Jun. 2017, pp. 328–339.

- [10] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: Vision and challenges," IEEE Internet of Things Journal (IoT-J), vol. 3, no. 5, pp. 637–646, 2016.
- [11] Leigh Johnson "Portable Computer Vision: TensorFlow 2.0 on a Raspberry Pi" online: <https://towardsdatascience.com/portable-computer-vision-tensorflow-2-0-on-a-raspberry-pi-part-1-of-2-84e318798ce9>.
- [12] R. Stahl, Z. Zhao, D. Mueller-Gritschneider, A. Gerstlauer, and U. Schlichtmann, "Fully distributed deep learning inference on resource-constrained edge devices," in International Conference on Embedded Computer Systems . Springer, 2019, pp. 77–90.
- [13] F. Iandola, "Exploring the Design Space of Deep Convolutional Neural Networks at Large Scale," no. December 2016, 2016.